자바스크립트 자료형과 형 변환

수업 녹음 정리 (10 월 26 일 2 부)

AI 활용 어플리케이션 개발자 과정 · 녹음 약 40 분 (음성 인식 오류는 문맥으로 바로잡음)

1. 왜 자료형을 배우는가

- React 나 Next.js 로 만드는 웹은 브라우저와 서버가 데이터를 주고받아야 함
- API 나 다운로드로 가져온 데이터가 어떤 형식인지 알아야 필요한 요소에 값을 넣고 자유롭게 다룰 수 있음
- 지금은 파이썬 기준으로 이해하고 있으므로, 파이썬은 이렇게 되고 자바스크립트는 이렇게 된다는 차이를 비교하며 학습하면 좋음

2. 숫자형 보충 (교재 25 쪽)

값	설명
`Infinity, -Infinity`	무한대, 마이너스 무한대
`NaN`	Not a Number. 숫자가 아닌 값. 에러처럼 보이지만 하나의 값으로 출력됨
`1 / 0`	Infinity 가 나옴. 파이썬은 ZeroDivisionError 에러가 났음

숫자와 문자를 연산하면

코드	결과
`"hello" + 2`	"hello2" (문자열 결합)
`"hello" * 2`	NaN (파이썬은 hellohello 로 반복됨)
`"hello" - 2`	NaN
`"hello" / 2`	NaN

저장하면 Prettier 가 세미콜론을 자동으로 붙여 줍니다. 파이썬과 달리 자바스크립트에서는 문자열 곱하기가 반복이 되지 않는다는 점이 핵심입니다.

3. 문자형 (교재 25 쪽 이후)

```
let myName = "이정환";
let welcomeText = "님 반가워요";
let result = myName + welcomeText;
console.log(result);   // 이정환님 반가워요
```

- 큰따옴표 " " 와 작은따옴표 ' ' 모두 문자열을 만들 수 있고, 백틱 ` ` 도 문자열을 만들 수 있음
- 문자열과 문자열을 +로 더하면 결합됨
- myName 처럼 두 단어는 카멜 표기법 (중간 대문자)
- 변수명은 이름만 봐도 용도를 유추할 수 있어야 좋은 이름. AI 가 만드는 변수명도 대체로 길고 설명적임. a, b 같은 이름은 쓰지 않음

var, let, const

키워드	설명
`var`	예전에 쓰던 선언 방식. 값이 무조건 바뀔 수 있어서 긴 코드에서 바꾸면 안 되는 변수를 실수로 바꿀 위험이 있음
`let`	값을 나중에 바꿀 수 있는 변수. 지금은 let 과 const 중 하나를 씀
`const`	바꾸면 안 되는 값. 바꾸려 하면 에러가 나서 실수를 막아 줌

템플릿 문자열 (백틱)

```
let names = '이정환';
let locations = '역곡';
let introduce = `${names}은 ${locations}에 살고 있습니다`;
console.log(introduce);    // 이정환은 역곡에 살고 있습니다
```

- 파이썬의 f 문자열 포맷팅과 같은 기능. 파이썬은 f"{name}" 처럼 쓰지만 자바스크립트는 백틱 하나로 씀
- 백틱(`)은 키보드 숫자 1 왼쪽의 키. Shift 를 누르지 않고 눌러야 하며, 작은따옴표와 모양이 비슷하지만 다른 기호
- 백틱 안에서 변수는 ${변수명} 형식으로 넣음 (달러 표시와 중괄호)
- 큰따옴표나 작은따옴표 안에서 ${}를 쓰면 변수가 아니라 글자로 인식되므로 반드시 백틱을 써야 함
- 백틱 안에서는 줄바꿈도 그대로 가능

강사가 name 대신 names, locations 같은 이름으로 바꾼 것은 name 이 브라우저에 이미 쓰이는 이름이라 충돌했기 때문으로 보이지만, 녹음이 흐려 확실하지 않습니다.

4. 불리언, null, undefined

```
let isSwitchOn = false;
let isSwitchOn2 = true;
let empty = null;
let notDefined;                // 값을 안 넣음 → undefined
console.log(typeof isSwitchOn);   // boolean
```

자료형	설명
`boolean`	true 또는 false. 파이썬은 True 와 False 로 첫 글자가 대문자였지만, 자바스크립트는 무조건 소문자
`null`	값이 없다고 의도적으로 표시한 상태. API 나 파이썬에서 결측값(데이터가 없는 값)을 만났을 때 자주 나옴
`undefined`	변수를 선언만 하고 값을 넣지 않은 상태. null 과 개념이 다름
`typeof`	값의 자료형을 확인하는 명령. 파이썬의 type()과 같은 역할

흔한 실수: "true"처럼 따옴표를 붙이면 불리언이 아니라 문자열입니다. typeof 로 찍어 보면 boolean 과 string 으로 구분됩니다.

isSwitchOn 처럼 세 단어 이상은 카멜 표기법으로 각 단어의 첫 글자를 대문자로 씁니다. 불리언 변수는 is 로 시작하는 이름을 많이 씁니다.

typeof null 의 결과가 object 로 나오는데, null 은 원시 값이지만 object 로 표시되는 자바스크립트의 특이한 동작입니다.

5. 형 변환 (교재 28~30 쪽)

묵시적(암묵적) 형 변환

```
let number = 10;
let string = '20';
const result = number + string;
console.log(result);   // '1020' (문자열)
```

- 숫자와 문자를 더하면 에러가 나지 않고 숫자가 자동으로 문자로 바뀌어 결합됨
- 이처럼 자동으로 형이 바뀌는 것을 묵시적 형 변환이라고 하고, 데이터 타입이 유동적이라는 뜻에서 다이나믹 타입이라고도 함

명시적(강제) 형 변환

코드	결과와 설명
`Number("10")`	10. 문자를 숫자로 바꿈 (파이썬의 int()에 해당). 함수 이름이 대문자 N 으로 시작
`Number("10 개")`	NaN. 숫자로 바꿀 수 없는 글자가 섞여서
`parseInt("10 개", 10)`	10. 앞쪽의 숫자만 읽어 변환. 두 번째 인수 10 은 10 진법을 뜻하며 생략해도 됨(물음표 옵션)
`String(2021)`	'2021'. 숫자를 문자로 바꿈 (파이썬의 str()에 해당)

parseInt 는 오류가 날 상황에서 숫자 부분만 살려서 대체값을 내는 변환이라고 이해하면 됩니다.

실습 (교재 30 쪽)

```
let varA;            // undefined
let varB = null;
let varC = true;
console.log(typeof varA, typeof varB, typeof varC);
// undefined, object, boolean

let strA = String(varA);
let strB = String(varB);
let strC = String(varC);
console.log(typeof strA, typeof strB, typeof strC);
// string, string, string
```

- 변환 전 자료형은 undefined, object, boolean 이었고, String() 함수로 바꾼 뒤에는 모두 string 이 됨
- null 이 object 로 나오는 것은 앞에서 말한 자바스크립트의 특이한 동작
- 이후 수업은 11 시 정각에 재개 (녹음은 여기서 끝)

6. 인식 오류를 고친 부분과 확인이 필요한 부분

원문	바로잡은 내용	근거
NAN, 나 디파인	NaN, undefined	숫자형, 불리언 설명
IMF, MIMF	Infinity, -Infinity	무한대 설명

원문	바로잡은 내용	근거
프리티어, 크리티어	Prettier	저장하면 세미콜론이 붙음
팩틱, 100 틱	백틱 (`)	키보드 위치 설명
폴스, 트루	false, true	불리언 설명
타이거, 타이머, 타입 오브	typeof	자료형 확인 설명
팔스일트, 팔스인트	parseInt	두 번째 인수 radix 설명

확인이 필요한 부분: 변수 이름을 names, locations 로 바꾼 이유(약 14 분), typeof 결과가 찍히지 않아 강사가 원인을 찾던 구간(약 25 분~27 분)은 녹음이 흐려 정리에서 제외했습니다. 코드 예제의 변수명은 녹음의 발음을 바탕으로 정리한 것이라 실제 실습 파일과 철자가 다를 수 있습니다.